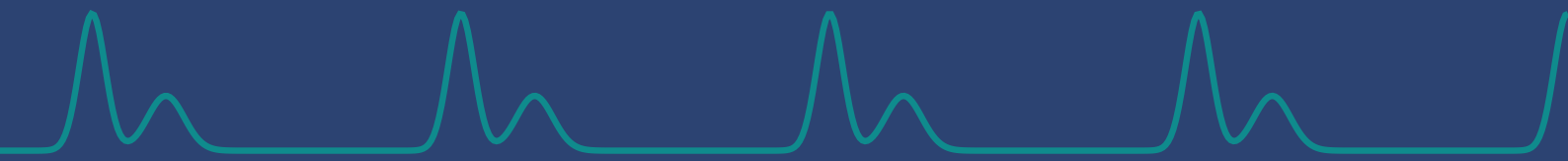
RAKSHPULSE

Hardware Integration Manual

MAX30102 + ESP32 / Arduino → rakshpulse software



A step-by-step handbook for the hardware integration team: wiring, firmware, installation, first test, recording, cuff references, analysis, troubleshooting and handoff.

Read this first

- RakshPulse is a research prototype, not a medical device.
- The reference firmware has not yet been compiled or run on a physical board.
- No model is validated for MAX30102 data.
- Every blood-pressure number on MAX30102 data is a demonstration, not a measurement.

Edition 1.0 for software release 5.1

RakshPulse software team

Contents

Click any entry to jump to it. PDF bookmarks give the same structure in your viewer's sidebar.

1. Introduction 4

Who this manual is for 4

What you will achieve 4

Project status, honestly 5

How to use this manual 5

2. The Technology 6

Photoplethysmography (PPG) 6

The MAX30102 sensor 7

The ESP32 board 8

3. Understanding the ML Model 10

The pipeline in plain words 10

Training, validation, inference, calibration and testing 11

Inference modes 11

The shipped model and its honest status 11

4. System Architecture 13

The data path, step by step 14

The reference path 14

5. Hardware Requirements and Wiring 15

Parts list 15

Wiring 15

6. Hardware Setup in 12 Steps 18

7. Software Installation 22

Windows (PowerShell) 22

Linux (bash) 23

Checking the installation: rakshpulse doctor 23

8. Command Reference 24

9. First Live Sensor Test 27

1. Find the port 27

2. Run the serial check 28

3. Two quick experiments worth doing . . . 29

10. Recording Data . . . 30

Record a session . . . 30

What happens during a recording . . . 31

The dataset folder . . . 31

11. Cuff Reference Collection . . . 33

Rules the software relies on . . . 33

Entering readings . . . 34

12. Analysis . . . 35

1. Validate the dataset . . . 35

2. Analyse a session . . . 35

3. Check signal quality . . . 36

Other analyze options . . . 36

13. Troubleshooting . . . 37

14. Handoff Checklist . . . 39

Checklist . . . 40

Sharing the dataset . . . 40

15. Limitations and Next Steps . . . 42

Limitations . . . 42

Next steps . . . 42

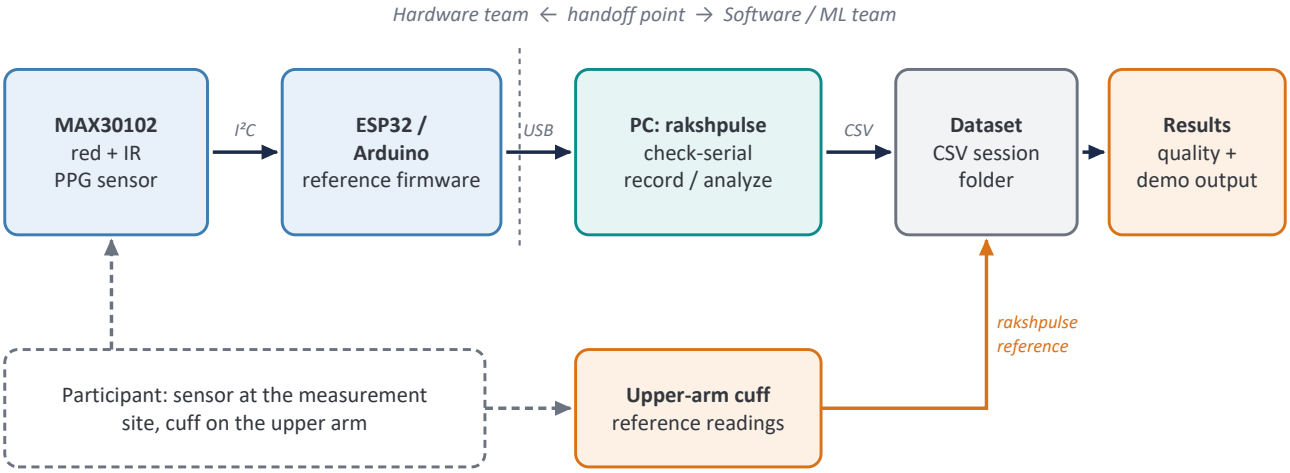
Where to find more . . . 43

01 Introduction

What RakshPulse is, what you will build, and how to use this manual

RakshPulse is research software that records photoplethysmography (PPG) signals from an optical sensor, checks their quality, extracts pulse features and runs blood-pressure (BP) estimation models. This manual takes you from a bare MAX30102 sensor and an ESP32 or Arduino board to a recorded, validated dataset that the software team can use.

Figure 1. RakshPulse at a glance



Signal path from the sensor to the analysis, and the cuff reference path. The dashed line marks the handoff point.

Who this manual is for

It is written for the hardware integration team: people who are comfortable with a soldering iron, a breadboard and the Arduino IDE, but who may never have used Python, a command line or a machine-learning model. Every command is printed in full. Every step says what you should see, and what to do if you do not.

What you will achieve

- Wire a MAX30102 sensor to an ESP32 (or another Arduino-compatible board).
- Upload the RakshPulse reference firmware, which streams raw red and infrared samples over USB.
- Install the `rakshpulse` software on Windows or Linux and confirm it works with `rakshpulse doctor`.
- Prove that the data stream is correct with `rakshpulse check-serial` (a PASS/FAIL test).
- Record five-minute sessions with cuff reference readings, validate them and run the analysis.
- Hand the dataset over privately so the software team can train a MAX30102-specific model.

Project status, honestly

> **!** **WARNING: read before you start**
>
> RakshPulse is a **research prototype**. It is **not a medical device** and must not be used to diagnose, monitor or treat anyone.
>
> - The software side is complete and tested with simulated sensors, replayed recordings and simulated serial ports. It has **never received data from a physical MAX30102**.
> - The reference firmware has **not been compiled or run on a board** by the software team (no board was available). You are the first to run it.
> - No RakshPulse model is validated for MAX30102 data. Every BP number shown for your device is labelled **demonstration**: it proves the pipeline runs. It is **not a blood-pressure measurement**.

That is why your work matters: the recordings you make are the first real MAX30102 data the project will have, and they are the only route to a model that could be trusted on this sensor.

How to use this manual

Chapters	Purpose	When to read
1–4	Background: PPG, the sensor, the board, the ML model and the system architecture	Once, before you start
5–7	Build: parts, wiring, firmware and software installation	While setting up
8–12	Use: commands, first test, recording, cuff readings, analysis	Every recording day
13	Troubleshooting: symptom → cause → solution	Whenever something fails
14–15	Handoff checklist, data sharing, limitations and next steps	Before you send data

Conventions

Commands you type appear in dark terminal blocks. A line starting with `PS>` is typed in Windows PowerShell; a line starting with `$` is typed in a Linux (or macOS) terminal. Lines without a prompt are output. Coloured boxes mark the kind of information:

TECHNICAL NOTE	Background or technical detail. Useful, but you can skip it on a first read.
IMPORTANT	Something you must do or confirm for the result to be valid.
WARNING	A safety, data-integrity or honesty issue. Do not ignore these.
EXPECTED RESULT / TIP	What you should see when a step works.

> **i** **TECHNICAL NOTE: COM5 is only an example**
>
> Throughout this manual `COM5` stands for *your* board's serial port. Windows may assign any COM number; on Linux the port is usually `/dev/ttyUSB0` or `/dev/ttyACM0`. Chapter 9 shows how to find yours.

02 The Technology

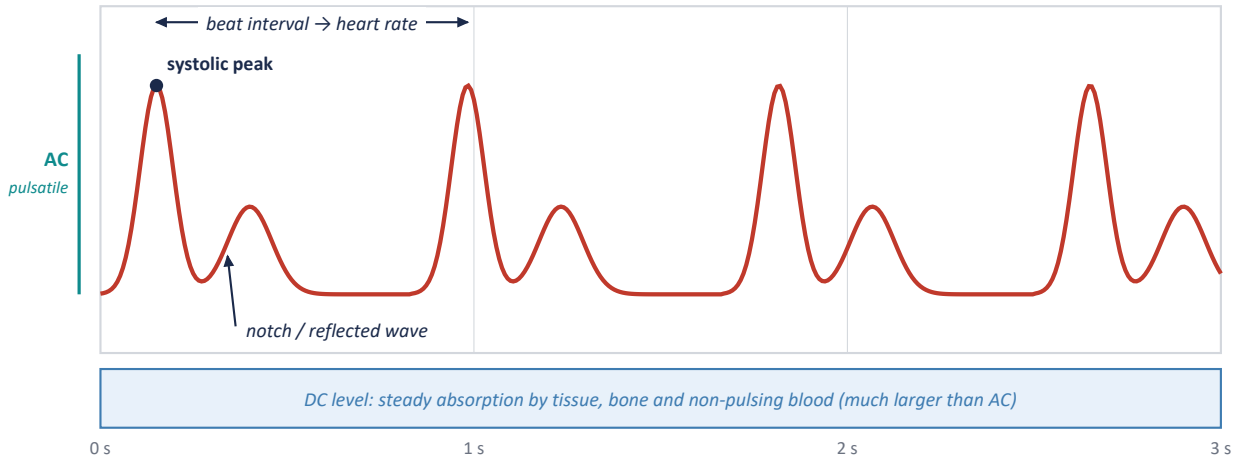
PPG, the MAX30102 sensor and the ESP32 board

Photoplethysmography (PPG)

Each heartbeat pushes a small extra volume of blood into the arteries of the skin. Blood absorbs light, so the amount of light that comes back out of the skin dips slightly with every beat. A PPG sensor shines light into the skin and measures the returning light many times per second. The result is a waveform that rises and falls with the heartbeat.

Figure 2. What a PPG signal looks like

Illustrative PPG shape (not recorded data); raw values arrive inverted or offset depending on the sensor



A small pulsatile (AC) wave rides on a large steady (DC) level. Heart rate comes from beat intervals; the model uses pulse shape.

- **DC component**: the large, steady level caused by tissue, bone and blood that does not pulse. It changes slowly with contact pressure, temperature and ambient light.
- **AC component**: the small pulsatile wave, typically only a small fraction of the DC level. This is the useful part.
- **Beat interval**: the time between successive peaks. Heart rate = 60 / interval (in seconds).
- **Pulse shape**: the rise time, width and the notch after the peak depend on how stiff the arteries are and on reflected pressure waves. Research suggests these shape features relate to blood pressure, which is why RakshPulse studies them.

i **TECHNICAL NOTE: why shape matters, and its limits**

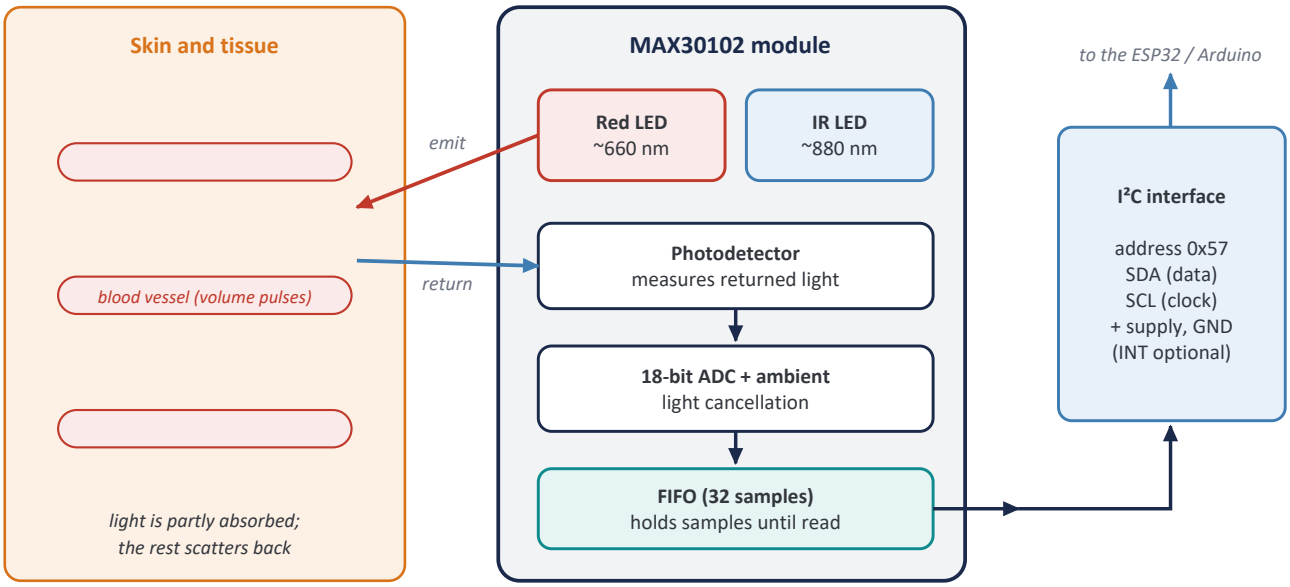
Blood pressure cannot be read directly from light. PPG-based BP estimation infers it statistically from pulse shape and timing, and published methods usually need per-person calibration with a cuff. Motion, contact pressure, skin tone, temperature and sensor placement all change the shape. That is why signal quality, consistent mounting and cuff references are so important in this project.

The MAX30102 sensor

The MAX30102 is an integrated pulse-oximetry and heart-rate sensor module originally from Maxim Integrated (now part of Analog Devices). In one small package it contains:

- a **red LED** (around 660 nm) and an **infrared (IR) LED** (around 880 nm);
- a **photodetector** that measures the light returning from the skin;
- an **analog-to-digital converter** (up to 18-bit resolution at the longest LED pulse width) with ambient-light cancellation;
- a **32-sample FIFO** buffer that stores samples until the microcontroller reads them;
- an **I²C interface** (7-bit address `0x57`) through which the board configures the sensor and reads samples.

Figure 3. Inside the MAX30102



Light from two LEDs enters the skin; the photodetector measures what returns. Samples queue in a 32-entry FIFO until the board reads them over I²C.

RakshPulse uses the sensor in **SpO2 mode** (`LED_MODE 2` in the firmware), which samples red and IR alternately. Both channels are recorded; the current feature set uses mainly IR.

Settings that matter, and what the firmware uses

Setting (firmware name)	Default in the sketch	What it controls
`SAMPLE_RATE`	100	Sensor samples per second (allowed: 50, 100, 200, 400, 800, 1000, 1600, 3200)
`SAMPLE_AVERAGE`	1	Samples averaged in the FIFO (1–32). Effective rate = SAMPLE_RATE / SAMPLE_AVERAGE
`PULSE_WIDTH`	411 µs	LED pulse width; 411 µs gives the full 18-bit range (0–262143)
`ADC_RANGE`	16384 nA	Full-scale current range of the ADC
`LED_POWER`	0x24	LED drive current (about 0.2 mA per step). Must be tuned for your site
`LED_MODE`	2	Red + IR (SpO2 mode)

> ★ **IMPORTANT: the sampling rate is never assumed**
> The software must be told the *effective* rate with `--fs`. With the defaults that is 100 / 1 = **100 Hz**. If you change `SAMPLE_RATE` or `SAMPLE_AVERAGE`, change `--fs` too. The firmware prints the value in its banner (`fs_hz=100`) and `record` refuses to start if the two disagree.

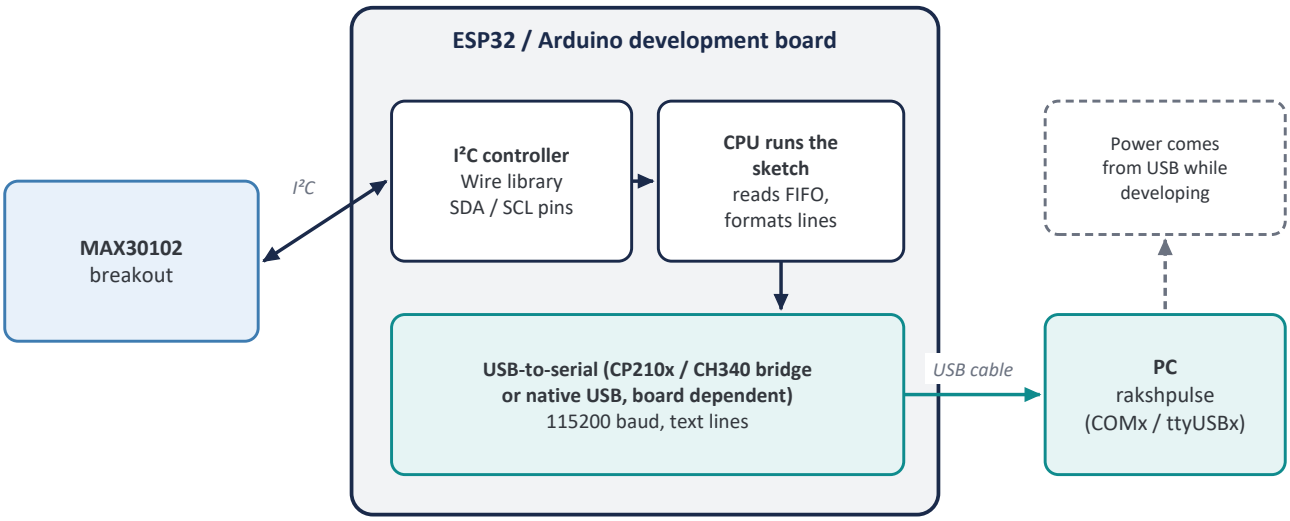
Power and electrical notes

The bare MAX30102 chip needs a low-voltage core supply and a separate LED supply. Most breakout boards include their own regulators and pull-up resistors so that they can be powered from a 3.3 V or 5 V pin, but **designs differ between vendors**. Use the supply pin and voltage your breakout's documentation specifies, and check that its I²C pull-ups are compatible with your board's logic level. This manual deliberately does not state a voltage, because the right value depends on the breakout you bought.

The ESP32 board

The ESP32 is a low-cost microcontroller from Espressif with built-in Wi-Fi and Bluetooth. It is sold on many development boards (DevKit, NodeMCU-style and others). In RakshPulse it has a simple job: read samples from the sensor over I²C and print them as text lines over USB. It does **no** filtering and **no** BP calculation; all processing happens on the PC, where it can be checked and repeated.

Figure 4. The ESP32 / Arduino board's job



Pin names and numbers: read your board's pinout (no pin numbers are assumed)

The board only moves data: it reads the sensor over I²C and prints text lines over USB serial.

- **I²C pins**: every board exposes an SDA (data) and SCL (clock) pin. Their GPIO numbers depend on the board. The sketch uses the board's default I²C pins unless you set `I2C_SDA_PIN` and `I2C_SCL_PIN`.
- **USB serial**: most ESP32 DevKit boards have a USB-to-serial bridge chip (commonly CP210x or CH340) that appears on the PC as a COM port. Some newer boards use native USB instead. You may need the bridge's driver.
- **Other boards**: any Arduino-compatible board with I²C and a USB serial port can run the sketch. The pin-override option only applies on ESP32; other boards always use their default I²C pins.

> **TECHNICAL NOTE: why not Wi-Fi or Bluetooth?**
> A wired USB connection is the simplest path with the fewest ways to lose samples. The software's serial reader expects a serial port; wireless streaming is not part of this release.

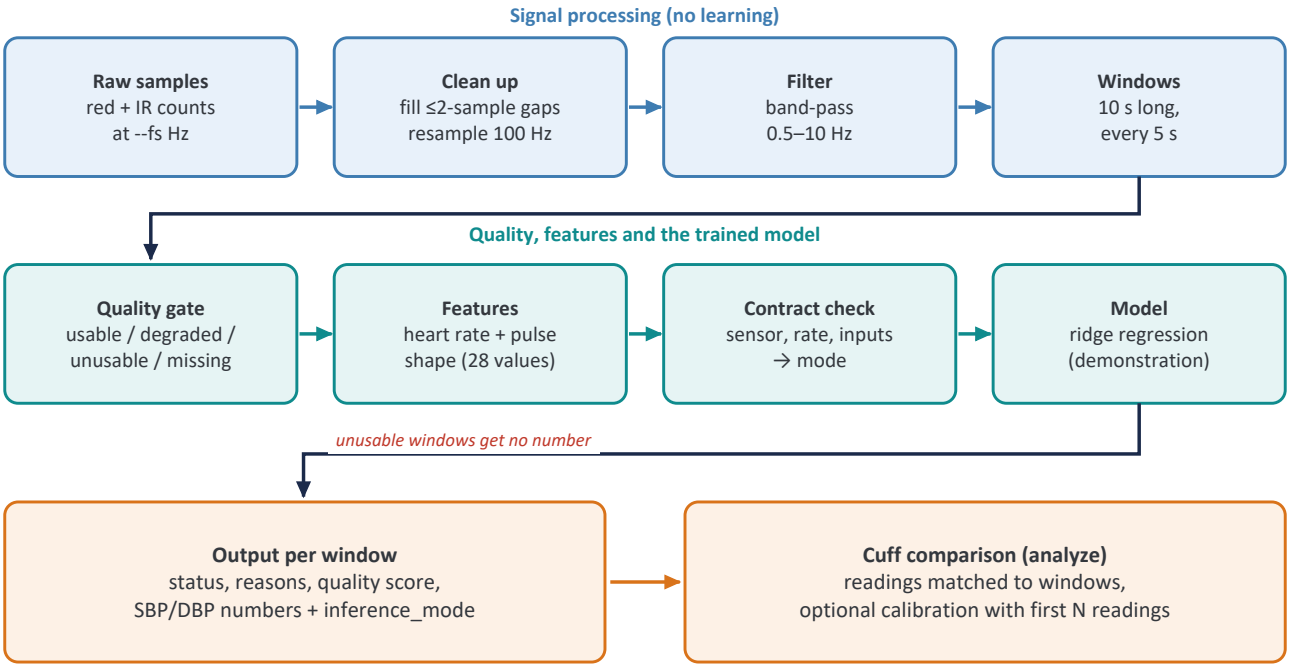
03 Understanding the ML Model

How a number is produced, and why it is not yet a measurement

You do not need to understand machine learning to integrate the hardware. This chapter explains just enough to read the outputs correctly, and to understand why your recordings and cuff readings matter.

The pipeline in plain words

Figure 5. The processing and ML pipeline



From raw counts to a mode-labelled output. Windows that fail the quality gate never receive a number.

- **Clean-up**: gaps of up to two samples are filled; longer gaps split the recording. The signal is resampled to 100 Hz if needed.
- **Filtering**: a band-pass filter (0.5–10 Hz) keeps the heartbeat and removes slow drift and fast noise.
- **Windows**: the signal is cut into 10-second windows, one starting every 5 seconds. A window never spans a gap.
- **Quality gate**: every window is labelled `usable`, `degraded`, `unusable` or `missing`, with the reasons (for example `rail_clipping` or `low_template_correlation`). Unusable and missing windows get no estimate.
- **Features**: heart rate and pulse-shape measurements (rise time, widths, ratios) are computed per window. The shipped model uses 28 of them.
- **Contract check**: before predicting, the software checks that the model supports this sensor, sampling rate and set of inputs. It decides the **inference mode** (see below) or refuses (exit code 4).
- **Model**: a ridge regression (a regularised linear model) turns the features into SBP and DBP numbers.

Training, validation, inference, calibration and testing

Term	Meaning in RakshPulse	Who does it
Training	Fitting the model's weights on recordings that have cuff readings (`rakshpulse train`).	Software team
Validation	Measuring error on people (and days) the model never saw during training. Results are compared with a 'predict the average' baseline; a model must beat it to be useful.	Software team
Inference	Running a trained model on new data to produce estimates (`analyze`, `infer`).	Both
Calibration	Using a person's first cuff readings to correct their later estimates (`analyze --calibrate N`). Calibration readings are never used to score accuracy.	Both
Testing	Software tests (164 automated tests) prove the code behaves as designed. They say nothing about BP accuracy.	Software team

Inference modes

Every model bundle carries a **contract**: the sensors, sampling rates and inputs it was built for, and the mode it runs in for each sensor. Every output row carries this mode, so it can never be mistaken later.

Mode	What it means	Exists for MAX30102 today?
`validated`	Trained and independently evaluated on this sensor type; accuracy is known.	**No.** No validated model exists for any sensor yet.
`experimental`	Trained on this sensor type, cross-validated only.	**No.** Needs your recordings first.
`demonstration`	The model was built for a different sensor or site. The numbers show that the pipeline runs and are **not** blood-pressure measurements.	**Yes**: this is what you will see.

The shipped model and its honest status

The default model is `models/auricle_ear_ridge_v2.joblib` (model id `auricle-ear-ridge`). It was trained on a public research dataset of PPG recorded at the **earlobe** with a different sensor. Its contract declares it **demonstration-only** for `max30102` and for the simulator.

What was measured (stored research results)	Model	Predict-the-average baseline
SBP error on people not seen in training (MAE, mmHg)	15.0	15.29
DBP error on people not seen in training (MAE, mmHg)	10.98	10.72

In words: on the earlobe dataset it was built from, the model is only marginally better than guessing the average for systolic pressure and slightly *worse* for diastolic. Its uncertainty interval (±31.4 mmHg SBP, ±21.8 mmHg DBP) is valid only for that earlobe data, so for MAX30102 the software reports the uncertainty as unavailable. Research on this project also found that what the PPG signal adds beyond a cuff reading did not carry over from one day to another.

> **!** **WARNING: never present demonstration numbers as blood pressure**
>
> Do not show, report, screenshot or record demonstration outputs as anyone's blood pressure. Do not compare them with a cuff and draw conclusions about accuracy. `analyze` prints the comparison only so that you can see the matching works; it adds *[demonstration mode: not an accuracy claim]* to every such line.

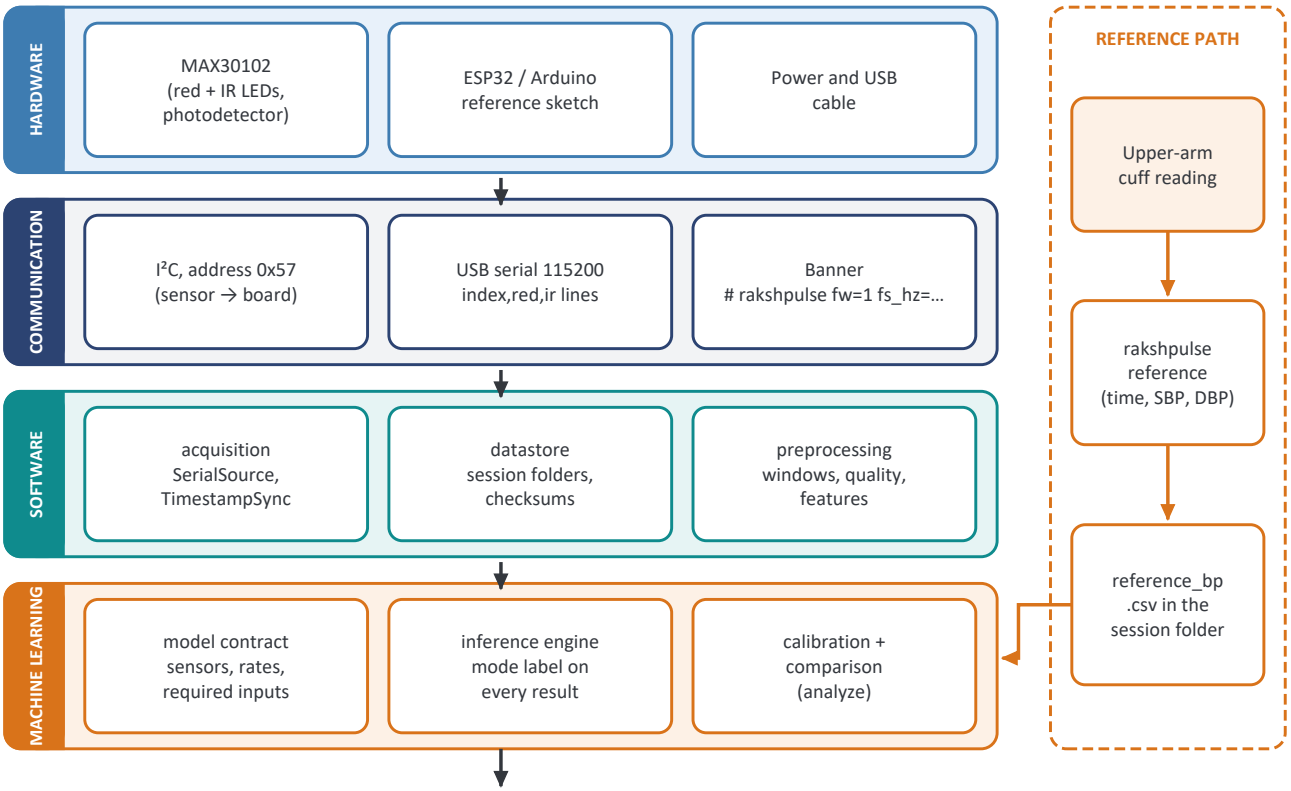
> **★** **IMPORTANT: what your data makes possible**
>
> A MAX30102-specific model needs recordings from this sensor with cuff readings, from several people and on more than one day. Only after it beats the baseline on unseen people and days could it run in `experimental` mode. That work starts when your dataset arrives (Chapter 15).

04 System Architecture

Hardware, communication, software, ML and reference paths

RakshPulse is built in layers. Each layer has one job and a defined interface to the next, so a problem can be located by checking the layers in order: if `check-serial` passes, the hardware and communication layers are working.

Figure 6. System architecture



Four layers and the separate reference path. Each layer only talks to the next one.

Layer	Components	Interface to the next layer	Owner
Hardware	MAX30102 breakout, ESP32/Arduino, USB cable	I²C at address 0x57	Hardware team
Communication	Reference sketch, USB serial at 115200 baud	Text lines `sample_index,red,ir` plus a `#` banner	Hardware team
Software	`acquisition` (serial reader, counter tracking), `datastore` (session folders, SHA-256 checksums), `preprocessing` (windows, quality, features)	CSV files in the dataset folder	Software team

Layer	Components	Interface to the next layer	Owner
Machine learning	Model contract, inference engine, calibration	Prediction CSV with `inference_mode`	Software team
Reference path	Upper-arm cuff, `rakshpulse reference`	`reference_bp.csv` with times in seconds	Hardware team (operator)

The data path, step by step

1. The sensor fills its FIFO at the configured rate.
2. The sketch reads the overflow counter, then the FIFO, and prints one line per sample. If samples were lost, the counter jumps by the number lost.
3. On the PC, `SerialSource` reads lines, skips `#` comments, counts malformed lines, and checks the device banner against `--fs`.
4. `TimestampSync` turns the counter into time (`sample_index / fs`), logs gaps, drops duplicates and records a device reboot as a gap instead of failing.
5. `DatasetStore` writes the session folder exclusively (it never overwrites) and hashes every file.
6. `analyze` validates the files, processes windows, checks the model contract, predicts, matches cuff readings and writes results.

The reference path

The cuff never connects to the computer. An operator reads the result from the cuff's display and types it in with `rakshpulse reference`, together with the time in seconds since the recording started. These readings are the 'ground truth' a future MAX30102 model will learn from, so their timing and accuracy matter as much as the PPG signal.

> **TECHNICAL NOTE: why the counter, not the PC clock?**
> The PC receives serial data in bursts, so arrival time is irregular. The sample counter advances exactly once per sample on the sensor, so `sample_index / fs` gives correct timing, and gaps in the counter reveal lost samples. The software also reports the drift between the counter and the PC clock (`drift_ppm`).

05 Hardware Requirements and Wiring

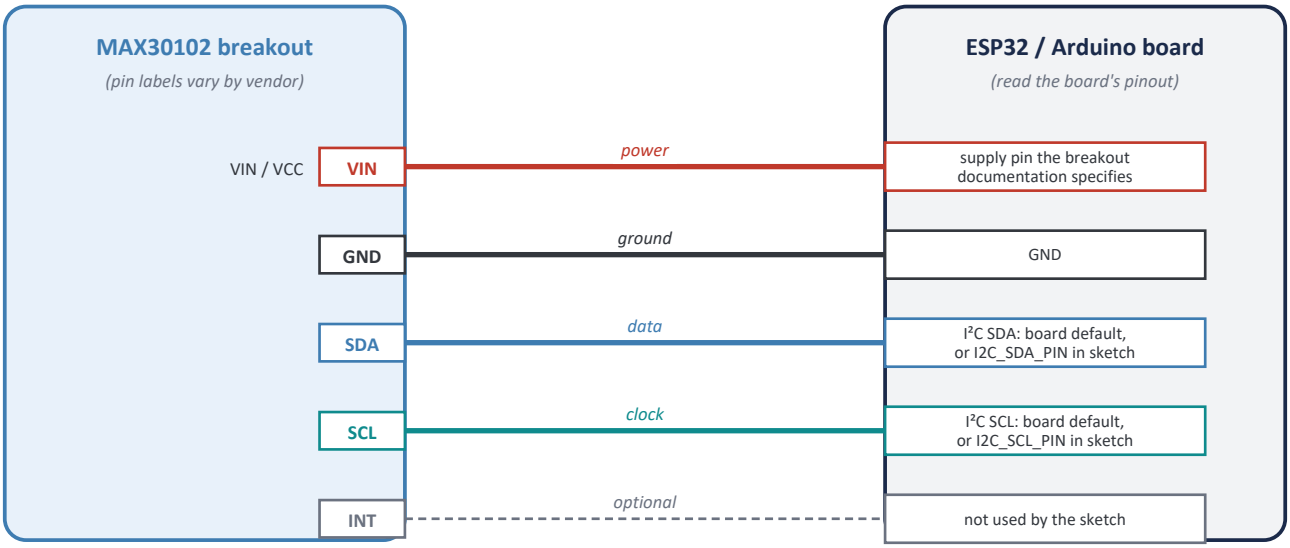
Parts list and the four essential connections

Parts list

Item	Requirement	Notes
MAX30102 breakout board	Any vendor; must expose supply, GND, SDA, SCL	Note the vendor and model in `--sensor-config` or `--notes`
Microcontroller board	ESP32 development board (recommended) or another Arduino-compatible board with I²C and USB serial	Must be supported by the Arduino IDE
USB cable	**Data** cable (not charge-only), suitable connector for the board	The most common cause of 'no ports found'
Jumper wires / soldered leads	Four connections (five if INT is wired)	Keep them short and secure; loose wires create noise
Mounting	Elastic band, clip or enclosure for the measurement site	Must hold steady, repeatable contact pressure
Computer	Windows 10/11 or Linux, Python 3.10–3.12, one free USB port	macOS works but is not covered here
Upper-arm BP cuff	An automatic cuff for reference readings	Record its make and model with every reading (`--device`)

Wiring

Figure 7. Wiring: which signals connect



Supply voltage, pull-up resistors and level shifting: follow your breakout's documentation.
This diagram shows which signals connect, not which pin numbers to use.

Connect power, ground, SDA and SCL. Pin numbers, supply voltage and pull-ups come from your board's and breakout's documentation.

Breakout signal	Connect to on the board	How to find it
Supply (VIN / VCC)	The supply pin your breakout's documentation specifies	Breakout datasheet or product page
GND	Any GND pin	Board pinout
SDA	The board's I²C SDA pin	Board pinout (often labelled SDA). On ESP32 you may choose another pin and set `I2C_SDA_PIN`
SCL	The board's I²C SCL pin	Board pinout (often labelled SCL). On ESP32 you may choose another pin and set `I2C_SCL_PIN`
INT	Not required	The sketch polls the FIFO and does not use the interrupt pin

> **!** **WARNING: no pin numbers are given on purpose**
> Default I²C pin numbers differ between ESP32 boards and between Arduino models, and breakout supply requirements differ between vendors. A wrong guess can damage a part. Look the values up for the exact board and breakout you have, write them down in your lab notebook, and record them in the session notes.

Finding your board's I²C pins

1. Look for pins printed **SDA** and **SCL** on the board itself.
2. Otherwise, open the pinout diagram from the board manufacturer's documentation for your exact model.
3. If you wire SDA/SCL to other pins on an ESP32, set both `I2C_SDA_PIN` and `I2C_SCL_PIN` in the sketch to those GPIO numbers. Leaving them at `-1` means 'use the board default'.
4. If the sketch prints `# ERROR: MAX30102 not found on I2C`, the pins, the supply or the wiring are wrong.

Mounting and placement

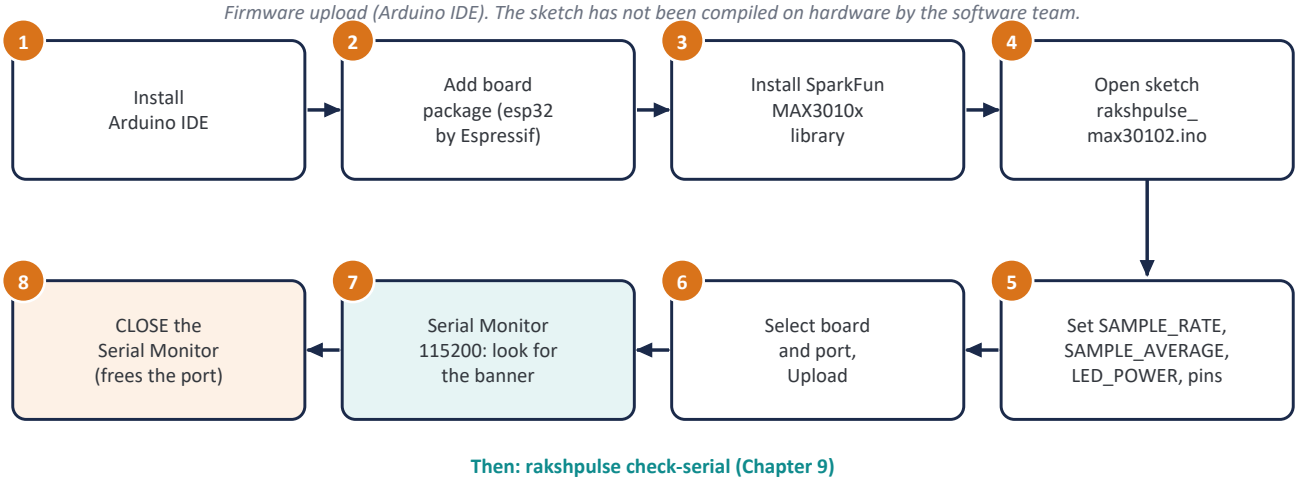
The project's default placement label is `behind_ear` (the `--placement` option). Whatever site you use, keep it the same for all sessions of a study, and describe the mounting in `--contact` (for example `"elastic band, moderate"`). Light leaking in from the side and changing pressure are the most common causes of poor windows. Shield the sensor from ambient light where possible.

06 Hardware Setup in 12 Steps

From a bare board to a streaming sensor

Do the steps in order. Each card says what to do, why, what you should see, the mistakes people commonly make, and what to try if it fails. Chapter 13 has the full troubleshooting table.

Figure 8. Uploading the firmware



Eight steps, in order. Close the Serial Monitor before running any rakshpulse command.

STEP 1 · Inspect the parts	
What to do	Check the breakout and the board for damaged pins or solder bridges. Identify the breakout's supply, GND, SDA and SCL pins.
Why	A damaged or mislabelled part wastes hours of debugging later.
Expected result	You know which breakout pin is which, and its required supply voltage.
Common mistakes	Assuming the pin order of a photo found online; different vendors use different layouts.
If it fails	Find the vendor's documentation; if none exists, consider a breakout that has it.

STEP 2 · Find the board's pinout	
What to do	Locate the SDA, SCL, GND and suitable supply pins of your exact board model.
Why	I²C pins differ between boards. The sketch defaults to the board's standard I²C pins.
Expected result	Written note of the four pins you will use.
Common mistakes	Using another board's pinout; confusing GPIO numbers with physical pin positions.
If it fails	Check the manufacturer's page for your exact model and revision.

STEP 3 · Wire the sensor (board unpowered)	
What to do	Connect supply, GND, SDA and SCL with short, secure wires. Leave INT unconnected.
Why	Wiring while powered risks shorts. Short wires reduce noise on the I²C bus.
Expected result	Four firm connections; nothing loose when the cable is moved.
Common mistakes	Swapping SDA and SCL; using a 5 V supply on a breakout that needs 3.3 V (or the reverse).
If it fails	Re-check each wire against your note from Step 2 with a multimeter (continuity).

STEP 4 · Install the Arduino IDE	
What to do	Install the Arduino IDE (or `arduino-cli`) from the official Arduino website.
Why	It compiles and uploads the sketch.
Expected result	The IDE opens.
Common mistakes	Installing an unofficial copy.
If it fails	Re-download from the official site.

STEP 5 · Add the ESP32 board package	
What to do	For ESP32: Boards Manager → install **esp32 by Espressif Systems**. Other boards: install their package.
Why	The IDE needs the board's compiler and upload tools.
Expected result	Your board model appears in Tools → Board.
Common mistakes	Skipping this and choosing a wrong board type.
If it fails	Restart the IDE; check the internet connection for the download.

STEP 6 · Install the sensor library	
What to do	Library Manager → install **SparkFun MAX3010x Pulse and Proximity Sensor Library** (header `MAX30105.h`, supports the MAX30102).
Why	The sketch uses it to configure the sensor and read the FIFO.
Expected result	The library is listed as installed.
Common mistakes	Installing a different MAX30102 library with another API: the sketch will not compile.
If it fails	Remove other MAX3010x libraries if the compiler picks the wrong header.

STEP 7 · Open the reference sketch	
What to do	Open `hardware/firmware/rakshpulse_max30102/rakshpulse_max30102.ino` from the RakshPulse repository.
Why	This sketch prints exactly the format the software expects.
Expected result	The sketch opens in its own folder named `rakshpulse_max30102`.
Common mistakes	Copying the code into a new sketch with another folder name, or editing the print format.
If it fails	Open the `.ino` file directly from the repository folder.

STEP 8 · Set the configuration	
What to do	Keep `SAMPLE_RATE 100` and `SAMPLE_AVERAGE 1` (100 Hz) unless you have a reason to change them. Set `I2C_SDA_PIN`/`I2C_SCL_PIN` only if you are not using the board's default pins.
Why	The effective rate must match `--fs` on the PC. 100 Hz is recommended.
Expected result	The values are noted in your lab notebook.
Common mistakes	Changing `SAMPLE_AVERAGE` and forgetting that the effective rate changes.
If it fails	Re-read the banner after upload: it states `fs_hz`.

STEP 9 · Select board and port, then upload	
What to do	Plug in the USB cable. Tools → Board: your model; Tools → Port: the new port. Click Upload.
Why	Compiles the sketch and writes it to the board.
Expected result	'Done uploading' (or similar) with no errors.
Common mistakes	Charge-only cable; wrong port; another program holding the port.
If it fails	Install the CP210x/CH340 driver; try another cable; some ESP32 boards need the BOOT button held while uploading starts.

STEP 10 · Look at the output, then close the monitor	
What to do	Open the Serial Monitor at **115200 baud**. Read a few lines. Then **close** the Serial Monitor.
Why	A quick visual check that the sensor was found. Only one program can use a port at a time.
Expected result	One banner line `# rakshpulse fw=1 fs_hz=100 ...`, then fast lines like `1234,95321,118204`.
Common mistakes	Leaving the Serial Monitor open: every `rakshpulse` command will then fail to open the port.
If it fails	Garbled text: wrong baud. `# ERROR: MAX30102 not found`: wiring, supply or pins.

STEP 11 · Tune the LED current	
What to do	With the sensor on the measurement site, watch the IR value. It should sit roughly in the middle of the range (about 30–70 % of 262143) and never reach 262143. Adjust `LED_POWER` and re-upload if needed.
Why	Too much light clips the signal (flat tops); too little gives a weak, noisy pulse.
Expected result	IR values well inside the range with a visible small oscillation at the heartbeat.
Common mistakes	Tuning on a fingertip and then recording at the ear: each site needs its own setting.
If it fails	Lower `LED_POWER` for clipping; raise it for a weak signal; shield ambient light.

STEP 12 · Mount the sensor	
What to do	Fix the sensor at the measurement site with the band or clip you will use for every session.
Why	Contact pressure changes the pulse shape. Repeatable mounting makes sessions comparable.
Expected result	The sensor does not move when the participant turns their head or speaks.
Common mistakes	Pressing too hard (flattens the pulse) or too lightly (motion artefacts).
If it fails	Use `rakshpulse check-serial` and later `inspect --quality` to compare mountings.

✓ **EXPECTED RESULT: hardware setup done**

The board streams lines and the Serial Monitor is closed. Continue with the software installation (Chapter 7) and the first live test (Chapter 9).

07 Software Installation

Windows and Linux, step by step

You need **Python 3.10, 3.11 or 3.12** and **git**. The installation creates a private Python environment (a *virtual environment*, `.venv`) inside the project folder, so it does not affect other software on the computer.

Windows (PowerShell)

1. Install Python 3.11 from python.org (tick *Add python.exe to PATH*) and Git for Windows.
2. Open **PowerShell** (Start → type *PowerShell*).
3. Run the commands below one at a time.

POWERSHELL · install

```
PS> git clone https://github.com/obstinix/RakshPulse.git
PS> cd RakshPulse
PS> py -3.11 -m venv .venv
PS> .\.venv\Scripts\Activate.ps1
PS> pip install -e ".[serial]"
PS> rakshpulse doctor
```

> **TECHNICAL NOTE: if Activate.ps1 is blocked**
> PowerShell may refuse to run the activation script. Run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, answer *Y*, then run the activation line again. This only affects your user account.

After activation the prompt starts with `(.venv)`. You must activate the environment again in every new PowerShell window before using `rakshpulse`:

POWERSHELL · every new window

```
PS> cd RakshPulse
PS> .\.venv\Scripts\Activate.ps1
```

USB driver and port on Windows

Open **Device Manager → Ports (COM & LPT)** and plug in the board. A new entry such as *Silicon Labs CP210x … (COM5)* or *USB-SERIAL CH340 (COM5)* should appear. If it shows under *Other devices* with a warning sign, install the CP210x or CH340 driver from the chip maker's website.

Linux (bash)

BASH · install

```
$ git clone https://github.com/obstinix/RakshPulse.git && cd RakshPulse
$ python3 -m venv .venv && source .venv/bin/activate
$ pip install -e ".[serial]"
$ sudo usermod -aG dialout "$USER"
$ rakshpulse doctor
```

★ **IMPORTANT: log out and back in once**

The `usermod` line gives your account access to serial ports (`/dev/ttyUSB0`, `/dev/ttyACM0`). It takes effect only after you log out and in again. Without it, opening the port fails with a permission error.

Checking the installation: rakshpulse doctor

POWERSHELL · real output from the development PC; your versions may differ; long lines wrapped for print: type each command on one line

```
PS> rakshpulse doctor
python 3.11.9 on Windows-10-10.0.26300-SP0
  ok   numpy 2.4.6
  ok   scipy 1.17.1
  ok   pandas 3.0.6
  ...
  ok   self-test: simulated 30 s -> 5 windows, HR error 0.3 bpm
  ok   default model auricle_ear_ridge_v2.joblib: auricle-ear-ridge runs on max30102 data
       in 'demonstration' mode
  ok   serial recording available (pyserial installed)
doctor: all required checks passed
```

✓ **EXPECTED RESULT**

The last line must be `doctor: all required checks passed`, and the serial line must say `serial recording available`. Optional packages (for example `torch`) may show `n/a`; that is fine.

doctor says	Meaning	Fix
`FAIL numpy` (or another required package)	Installation incomplete	Activate `.venv`, rerun `pip install -e ".[serial]"`
`n/a serial recording needs: pip install -e .[serial]`	pyserial missing	`pip install -e ".[serial]"`
`FAIL default model`	The model file is missing or damaged	Re-clone the repository; do not copy model files from elsewhere
`rakshpulse` is not recognised	Environment not activated	Run the activation line, then retry

08 Command Reference

The eight commands you will use

Every command has built-in help: add `-h`, for example `rakshpulse record -h`. Every command ends with an **exit code** that scripts can check:

Exit code	Meaning
0	Success
1	A check failed, or there is no trustworthy result
2	Invalid usage or input (a typo, a missing option, a bad file)
3	Device or serial problem (port missing, no data, wrong rate)
4	Incompatible model (sensor, rate or inputs not supported)

rakshpulse doctor: Check the installation

Checks Python, required and optional packages, runs a 30 s simulated self-test, loads the default model and reports its mode for MAX30102.

Options: none.

EXAMPLE

```
$ rakshpulse doctor
```

rakshpulse models: List models and what they support

Shows each model bundle, the mode it would run in for your sensor, its required inputs and channels.

Main options: `--sensor` (default `max30102`), `--fs`, `--model FILE`, `--models-dir`, `--verbose` (full contract and limitations)

EXAMPLE

```
$ rakshpulse models --sensor max30102 --fs 100 --verbose
```

rakshpulse ports: Find the board's serial port

Lists serial ports with a description. Run it with the board unplugged and plugged in; the new entry is your board.

Options: none.

EXAMPLE

```
$ rakshpulse ports
```

rakshpulse check-serial: Test the live data stream

Reads for a few seconds and checks the contract: valid lines, counter, rate, banner, channel ranges. Prints PASS or FAIL.

Main options: `--port` (required), `--fs` (required), `--baud` (115200), `--seconds` (10), `--timeout` (5), `--fields`, `--delimiter`

EXAMPLE

```
$ rakshpulse check-serial --port COM5 --fs 100
```

rakshpulse record: Record a session

Writes a new session folder; never overwrites. Stops at `--duration`, Ctrl-C, or after `--timeout` seconds without data.

Main options: `--source serial`, `--port`, `--fs`, `--out`, `--subject`, `--session`, `--duration`, `--placement`, `--contact`, `--notes`, `--sensor-config`, `--timeout`, `--baud`

EXAMPLE · long lines wrapped for print: type each command on one line

```
$ rakshpulse record --source serial --port COM5 --fs 100 --out data/hw --subject P001
      --session D1 --duration 300
```

rakshpulse inspect: List and validate a dataset

Lists sessions; with `--validate` it must say `validation: all PASS`.

Main options: `--dataset` (required), `--validate` (checksums, schema, ordering, ranges, references), `--quality` (window quality summary)

EXAMPLE

```
$ rakshpulse inspect --dataset data/hw --validate
```

rakshpulse reference: Add a cuff reading

Appends one reading to `reference_bp.csv` and updates the checksum. Readings are never edited in place.

Main options: `--dataset`, `--subject`, `--session`, `--time` (s from recording start), `--sbp`, `--dbp`, `--hr`, `--device`

EXAMPLE · long lines wrapped for print: type each command on one line

```
$ rakshpulse reference --dataset data/hw --subject P001 --session D1 --time 60 --sbp 121
      --dbp 79
```

rakshpulse analyze: Process and run the model

Validates, processes windows, checks the model contract, predicts, matches cuff readings, writes `derived/` files.

Main options: `--dataset`, `--subject`, `--session`, `--model`, `--sensor`, `--demographics`, `--calibrate N`, `--tolerance` (30 s), `--force`, `--overwrite`

EXAMPLE

```
$ rakshpulse analyze --dataset data/hw --subject P001 --session D1
```

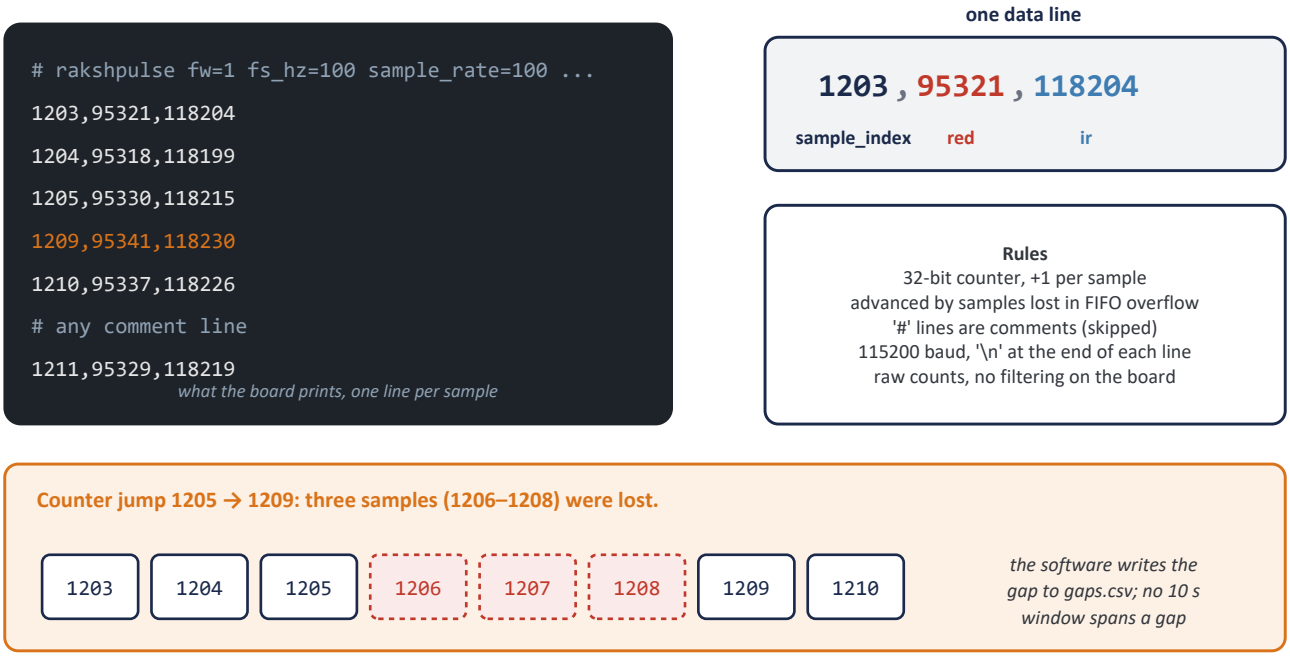
> **TECHNICAL NOTE: other commands**
> The CLI also has `simulate`, `preprocess`, `features`, `train`, `calibrate`, `evaluate`, `infer` and `export`. They are used by the software team and described in `docs/USER_GUIDE.md`. `rakshpulse infer --source serial --port COM5 --fs 100 --out live.csv` gives a live view, but it is not needed for the handoff.

09 First Live Sensor Test

Finding the port and passing check-serial

This is the moment the hardware and software meet for the first time. Close the Arduino Serial Monitor, activate the environment, and place a finger (or the measurement site) on the sensor.

Figure 9. Serial communication contract



One text line per sample. The counter lets the software detect lost samples instead of silently joining time.

1. Find the port

POWERSHELL · illustrative output; descriptions depend on your board

```
PS> rakshpulse ports
device                                  description  hwid
  COM5  Silicon Labs CP210x USB to UART Bridge (COM5)  USB VID:PID=...
```

Run `rakshpulse ports` once with the board unplugged and once plugged in: the new line is your board. On Windows, Bluetooth serial ports (*Standard Serial over Bluetooth link*) are not your board. If you see `no serial ports found`, check the cable (data, not charge-only) and the driver.

2. Run the serial check

POWERSHELL · replace COM5 with your port

```
PS> rakshpulse check-serial --port COM5 --fs 100
```

On Linux use the device path, for example `rakshpulse check-serial --port /dev/ttyUSB0 --fs 100`. The check reads for 10 seconds (change with `--seconds`) and prints one line per measurement, then a verdict:

Report field	What it means	Good value
`samples`	Valid data lines received	About fs × seconds (≈ 1000 for 10 s at 100 Hz)
`malformed_lines`	Lines that did not match `sample_index,red,ir`	0
`duplicates`	Repeated sample indices (dropped)	0
`missing_samples`, `gaps`	Samples lost (counter jumps)	0 (occasional small gaps are logged, not fatal)
`counter_resets`	The board rebooted during the check	0
`measured_rate_hz`	Rate measured from arrival times	Close to `--fs`
`declared_fs_hz`	Rate the firmware banner declares	Equal to `--fs`
`device_info`	Parsed banner fields (fw, fs_hz, LED settings…)	Present
`red_range`, `ir_range`	Minimum and maximum of each channel	Inside 0–262143, not constant, not at 262143
`ok`	Overall result	`True`

✓ **EXPECTED RESULT: PASS**

The last line reads `serial check: PASS (data matches the RakshPulse contract)` and the exit code is 0. Your hardware and communication layers work. Continue to Chapter 10.

! **WARNING: FAIL**

The report ends with one or more `problem:` lines and `serial check: FAIL (see docs/HARDWARE_HANDOFF.md, Troubleshooting)`. The exit code is 3 if no samples arrived, otherwise 1. Look the problem up in Chapter 13, fix it and run the check again. **Do not record until the check passes.**

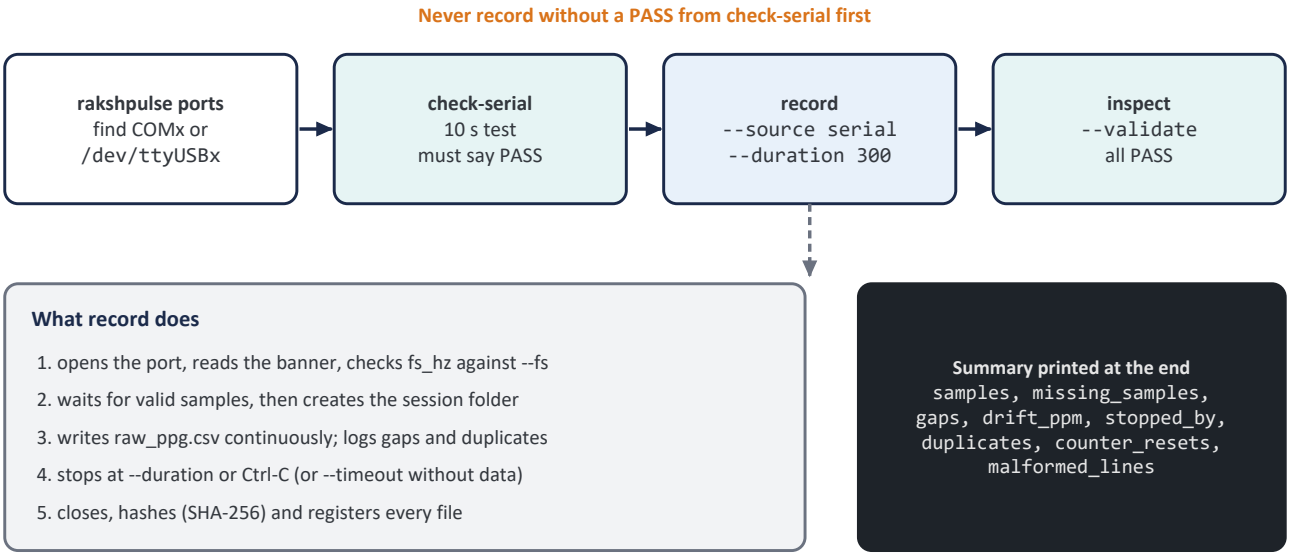
3. Two quick experiments worth doing

- **Remove the finger** during a check: the IR range should collapse towards a flat value. This confirms the sensor reacts to tissue.
- **Verify lost-sample detection**: temporarily add a short `delay()` in the sketch's `loop()`, upload, and run the check. It should report `missing_samples` > 0. Remove the delay afterwards and re-upload. This confirms the overflow counter works on your board (the software team could not test it on hardware).

10 Recording Data

Sessions, files and what they contain

Figure 10. Recording workflow



Check, record, validate: the order matters.

Record a session

POWERSHELL · 5-minute recording; long lines wrapped for print: type each command on one line

```
PS> rakshpulse record --source serial --port COM5 --fs 100 --out data/hw --subject P001
      --session D1 --duration 300
recording P001/D1 at 100 Hz from serial (Ctrl-C to stop)
```

A fuller version records the placement, the contact conditions and the sensor settings with the session. These details matter later, when recordings are compared:

POWERSHELL · recommended form; long lines wrapped for print: type each command on one line

```
PS> rakshpulse record --source serial --port COM5 --fs 100 --out data/rakshpulse_hw --subject
      P001 --session D1 --duration 300 --placement behind_ear --contact "elastic band,
      moderate" --notes "breakout vendor X, LED_POWER 0x24"
```

Option	Meaning
`--out`	The dataset folder. Use one folder per study, e.g. `data/rakshpulse_hw`.
`--subject`	A pseudonymous ID such as `P001`. **Never a name or initials.**
`--session`	Session label. Use `D1`, `D2` … for days; a new label for each new recording.

Option	Meaning
`--duration`	Seconds to record. 300 s (5 min) is recommended; at least 30 s is needed for any analysis.
`--placement`	Measurement site label (default `behind_ear`).
`--contact`	Free text: mounting and pressure.
`--notes`	Free text; the firmware banner is added automatically.
`--sensor-config`	Optional JSON with sensor settings, e.g. `'{"sample_rate_hz": 100, "pulse_width_us": 411}'`.
`--timeout`	Seconds without valid data before the recording ends (the data already received is kept).

What happens during a recording

- `record` checks the firmware banner. If the banner's `fs_hz` differs from `--fs`, it refuses to start.
- The session folder is created only after valid samples arrive, and **exclusively**: if `P001/D1` already exists, the command stops. Nothing is ever overwritten. Use a new `--session` label instead.
- Stopping with `--duration` or **Ctrl-C** both close, hash and register the files. Unplugging the board ends the recording after `--timeout` seconds; the samples received are kept.
- At the end a summary is printed: `samples`, `missing_samples`, `gaps`, `drift_ppm`, `stopped_by`, `duplicates`, `counter_resets` and `malformed_lines`.

★ **IMPORTANT: keep the participant still and the setup constant**
Seated, resting, not talking, the sensor mounted the same way each time. Write down anything unusual in `--notes`. A recording with lots of motion produces mostly `unusable` windows.

The dataset folder

Figure 11. Dataset folder after recording and analysis



One folder per subject and session. Share the whole dataset folder, zipped, privately.

File	Contents
`raw_ppg.csv`	`timestamp` (s), `subject_id`, `session_id`, `sample_index`, `red`, `infrared` (raw counts)
`session_metadata.csv`	Date, `sensor_type` (`max30102`), placement, `sampling_rate`, contact, notes (with the banner), sensor config, software version, `simulated`
`gaps.csv`	`timestamp`, `first_missing_index`, `n_missing`, `detected_by` (`counter`, `counter_reset`, …)
`reference_bp.csv`	Cuff readings: `timestamp` (s), `sbp`, `dbp`, `heart_rate`, `measurement_device`
`derived/`	Results of `analyze`: `windows.csv`, `segments.csv`, `predictions_<model>.csv`
`dataset_manifest.csv`	SHA-256 checksum of every file; `inspect --validate` uses it to detect changes
`subjects.csv`	One row per subject ID

! **WARNING: never edit the CSV files by hand**

Editing a file breaks its checksum and `inspect --validate` will fail. If something was recorded wrongly, record a new session and describe the problem in its notes.

¡ **TECHNICAL NOTE: try it without hardware first**

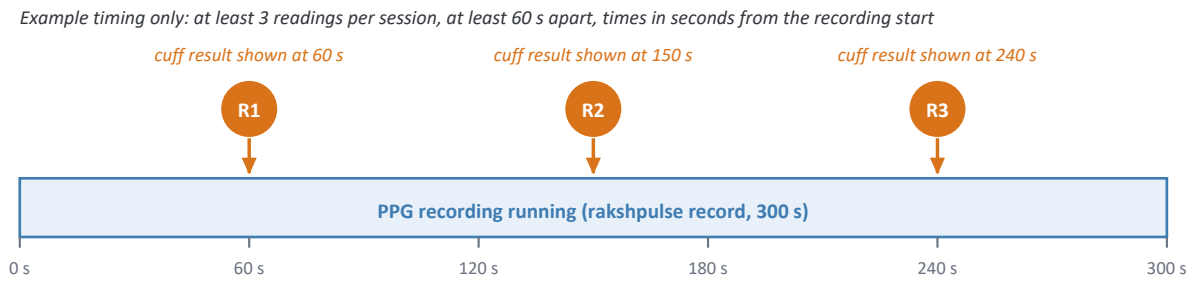
The repository contains a complete **simulated** example session in `hardware/examples/example_dataset/` (subject `SIM001`, session `D1`, 90 s, two simulated cuff readings). Copy the folder somewhere else and run `inspect` and `analyze` on the copy to see what real outputs look like. Simulated data is labelled `simulated=True` everywhere and says nothing about blood pressure.

11 Cuff Reference Collection

Timing and entering reference readings

A cuff reading is the reference the PPG is compared with. Without cuff readings a recording can show signal quality, but it can never help train or evaluate a BP model.

Figure 12. Cuff reference workflow




```
rakshpulse reference --dataset data/hw --subject P001 --session D1 --time 60  --sbp .. --dbp ..
rakshpulse reference --dataset data/hw --subject P001 --session D1 --time 150 --sbp .. --dbp ..
rakshpulse reference --dataset data/hw --subject P001 --session D1 --time 240 --sbp .. --dbp ..
```

values typed from the cuff display

Each reading is entered with its time in seconds from the start of the recording.

Rules the software relies on

- Take **at least 3 readings per session**, **at least 60 s apart**.
- The reading time is **seconds from the start of the recording**, at the moment the cuff **shows its result**. Start a stopwatch together with `record`, or note the clock time when recording starts.
- Each reading is matched with PPG windows within `--tolerance` seconds (default 30 s) in `analyze`.
- Allowed ranges: SBP 50–260 mmHg, DBP 25–160 mmHg, and SBP must be greater than DBP. Values outside are rejected.
- Readings are appended, never edited. A wrong entry stays in the file; note it and inform the software team.

Entering readings

During the recording, write each reading on paper: the time in seconds, SBP, DBP and heart rate. When the recording has finished, enter them one by one:

POWERSHELL · example values; type the numbers your cuff shows; long lines wrapped for print: type each command on one line

```
PS> rakshpulse reference --dataset data/rakshpulse_hw --subject P001 --session D1 --time 60
      --sbp 121 --dbp 79 --hr 72 --device "Omron M3"
PS> rakshpulse reference --dataset data/rakshpulse_hw --subject P001 --session D1 --time 150
      --sbp 118 --dbp 77 --hr 70 --device "Omron M3"
PS> rakshpulse reference --dataset data/rakshpulse_hw --subject P001 --session D1 --time 240
      --sbp 119 --dbp 78 --hr 71 --device "Omron M3"
```

★ **IMPORTANT: what this manual does not define**

This manual does not prescribe a clinical measurement protocol (posture, rest period, arm, cuff size, which arm relative to the sensor). Follow `docs/DATA_COLLECTION_PROTOCOL.md` in the repository and any protocol approved for your study. Record the cuff make and model in `--device` every time.

Any study with human participants needs informed consent and the approvals your institution requires.

i **TECHNICAL NOTE: why timing matters**

BP changes from minute to minute. If a reading is entered with the wrong time, it is compared with the wrong windows, and a future model learns from wrong pairs. A careful time within a few seconds is far better than an approximate one.

12 Analysis

Validating, analysing and reading the results

1. Validate the dataset

POWERSHELL

```
PS> rakshpulse inspect --dataset data/rakshpulse_hw --validate
...
validation: all PASS
```

This checks every file against its checksum, the column schema, the ordering of samples, the value ranges and the reference readings. Anything other than `validation: all PASS` lists the failing checks.

2. Analyse a session

POWERSHELL

```
PS> rakshpulse analyze --dataset data/rakshpulse_hw --subject P001 --session D1
```

The following is **real output** from the simulated example session (`SIM001`), so you know what to expect. For a MAX30102 session the first line shows `max30102` instead of `simulator`:

OUTPUT · real output, simulated example; long lines wrapped for print: type each command on one line

```
[1/5] validating SIM001/D1 (simulator, 100 Hz)
[2/5] processing windows
[3/5] model checks and prediction
      model auricle-ear-ridge (auricle-ear-ridge-1+contract1): mode=demonstration
      DEMONSTRATION MODE: this model has not been validated for this sensor; the numbers
      show that the pipeline runs and are NOT blood-pressure measurements.
      windows: 17 total, 17 predicted; status counts {'usable': 14, 'degraded': 3}
[4/5] cuff references
      2 independent readings vs population estimate: SBP MAE 15.6 ... [demonstration
      mode: not an accuracy claim]
[5/5] writing results
      median estimate over accepted windows: 102/65 mmHg  (uncertainty unavailable for
      sensor 'simulator' ...)
      wrote ...\sessions\SIM001\D1\derived\predictions_auricle-ear-ridge.csv
```

Step	What it does
[1/5] validating	Same checks as `inspect --validate`, for this session. Stops on failure unless `--force`.
[2/5] processing windows	10 s windows every 5 s, quality labels and reasons, features.
[3/5] model checks and prediction	Contract check (sensor, rate, inputs) → mode; predictions for usable and degraded windows.

Step	What it does
[4/5] cuff references	Matches readings with windows; with `--calibrate N` the first N readings calibrate and the rest are compared independently.
[5/5] writing results	Writes `derived/windows.csv`, `derived/segments.csv`, `derived/predictions_<model>.csv`.

> **!** **WARNING: reading the analyze output**
> The `median estimate … mmHg` and `MAE` lines are demonstration outputs. They show that windows, matching and export work. They are not the participant's blood pressure and not the accuracy of anything.

3. Check signal quality

POWERSHELL · real output for the simulated example; long lines wrapped for print: type each command on one line

```
PS> rakshpulse inspect --dataset data/rakshpulse_hw --quality
subject_id session_id  windows usable degraded  top_reasons
    SIM001         D1       17    82%      18%  filled_1_missing_samples (4);
      margin:spike_frac (2); ...
```

Window status	Meaning	Typical causes
`usable`	Passed every quality check	Good contact, participant still
`degraded`	Passed, but close to a limit (reasons start with `margin:`) or had tiny filled gaps	Slight motion, a few lost samples
`unusable`	Failed a check; no estimate	`rail_clipping` (LED too strong), `low_template_correlation` or `irregular_beat_intervals` (motion, poor contact)
`missing`	No signal	`no_signal_flatline`: sensor off the skin, LED current 0, wiring

> **✓** **EXPECTED RESULT: target for the handoff**
> For an on-body recording at rest, aim for **at least 80 % usable** windows. Fewer means the mounting, LED current or motion needs attention before more data is collected.

Other analyze options

- `--calibrate 1`: use the first cuff reading to calibrate; the remaining readings are compared independently. Calibration windows are never scored.
- `--overwrite`: replace an earlier predictions file for the same model (raw data is never touched).
- `--model FILE`: use another bundle. `rakshpulse models --sensor max30102 --fs 100` shows which ones are compatible.
- `--demographics age=35,sex=1,height_cm=172,weight_kg=70`: only for models that require demographics. The default model does not.

13 Troubleshooting

Symptom → cause → solution

Find the message or behaviour you see in the left column. Work through the causes in order: the first one is the most common.

Symptom	Likely cause	Solution
`rakshpulse` is not recognised	Virtual environment not activated	Activate `.venv` (Chapter 7) in this window
`error: ... needs the optional 'pyserial' package`	Installed without the serial extra	`pip install -e ".[serial]"`
`no serial ports found`	Board unpowered, charge-only cable, missing driver	Use a data cable; install the CP210x/CH340 driver; try another USB port
`cannot open serial port 'COM5'`	Wrong port; port held by the Serial Monitor; Linux permissions	`rakshpulse ports`; close the Serial Monitor; join `dialout` and log in again
Upload fails in the Arduino IDE	Wrong board/port; port busy; boot mode	Select the right board and port; close other programs; hold BOOT while the upload starts (some ESP32 boards)
`# ERROR: MAX30102 not found on I2C`	Wiring, supply or I²C pins wrong	Check each wire; check supply per the breakout docs; set `I2C_SDA_PIN`/`I2C_SCL_PIN` if not using default pins
`no valid samples ... for 5 s`	Baud mismatch; firmware not running; sensor not found	Use 115200 on both sides; re-upload; read the Serial Monitor for `# ERROR` lines
Many `malformed_lines`	Baud mismatch (garbled text); extra debug prints; different format	Match the baud; remove prints from the sketch; adjust `--fields`/`--delimiter`
`firmware declares fs_hz=X but --fs is Y`	`--fs` does not match the sketch	Pass `--fs X`, or change `SAMPLE_RATE`/`SAMPLE_AVERAGE` and re-upload
`measured rate ... differs from --fs`	Board cannot keep up; averaging not accounted for	Lower the rate or raise the baud (both sides); recompute fs = SAMPLE_RATE / SAMPLE_AVERAGE
`missing_samples` / gaps	FIFO overflow: loop or serial too slow	Remove `delay()` from `loop()`; raise the baud on both sides. Gaps are logged; windows never span them
`counter_resets` > 0	The board rebooted (power dip, loose cable)	Secure the cable and power; the reset is recorded as a gap
A channel is constant / all windows `missing (no_signal_flatline)`	Sensor not on the skin; LED current 0; wiring	Check contact and `LED_POWER`; re-check wiring
`unusable` with `rail_clipping`	Too much light: LED current too high or ambient light	Lower `LED_POWER`; shield the sensor

Symptom	Likely cause	Solution
`unusable` with `low_template_correlation` / `irregular_beat_intervals`	Motion or poor contact	Keep still; improve the mounting
`recording too short`	Less than 30 s of contiguous data	Record longer; each window needs 10 s without a gap
`timestamps imply about X Hz but ... declared as Y Hz`	Recorded with the wrong `--fs`	Re-record with the correct `--fs`
Session already exists	`record` never overwrites	Use a new `--session` label
`validation` fails after copying files	A file changed (edited, line endings converted)	Do not edit CSVs; copy folders as-is (zip them)
`model not usable for this input` (exit 4)	Unsupported sensor or rate, or missing demographics	`rakshpulse models --sensor max30102 --fs 100 --verbose`
`requires demographic inputs`	The chosen model needs them	Add `--demographics age=...,sex=...,height_cm=...,weight_kg=...`, or use the default model
`no trustworthy prediction`	No window passed the quality and input checks	Fix the signal first; see `inspect --quality`
Model fails to load	Bundle from an unknown source	Use only bundles from this repository; joblib files from unknown sources are unsafe

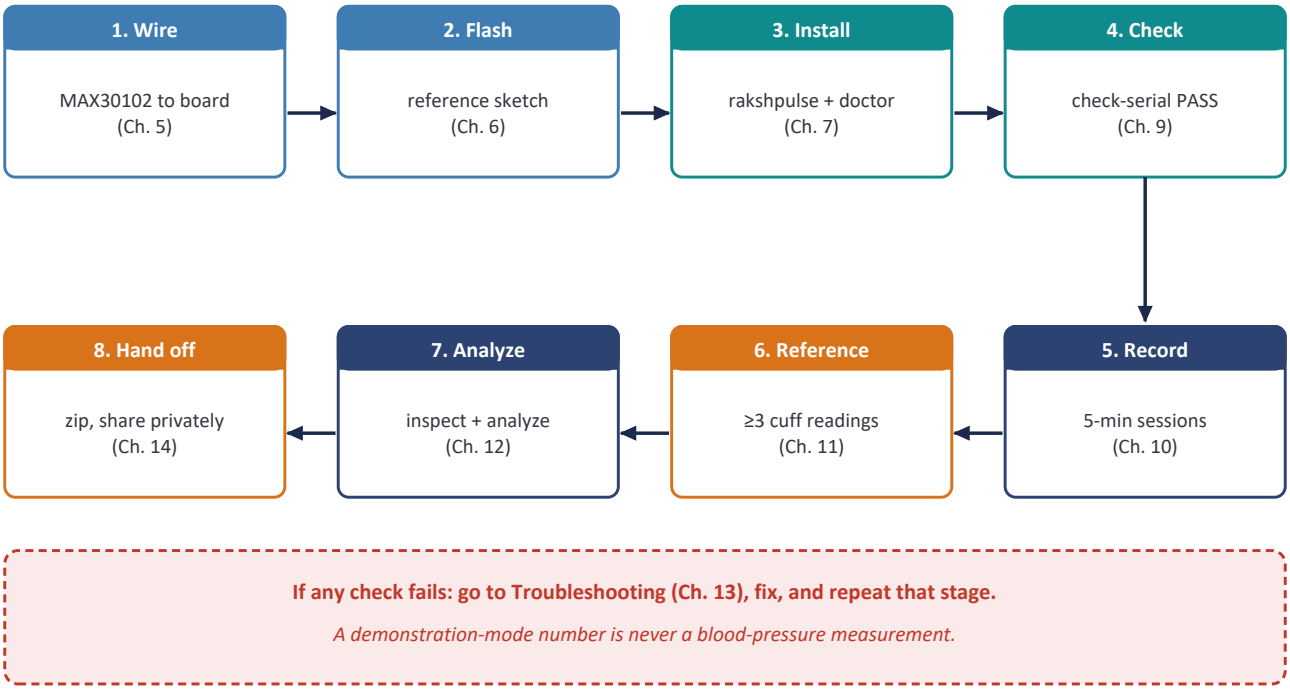
✓ **TIP: still stuck?**

Collect: the exact command, its full output, the exit code (PowerShell: `$LASTEXITCODE`; bash: `echo $?`), the first 20 lines from the Serial Monitor, your board and breakout model, and the sketch settings. Send these to the software team; they can usually diagnose from that alone.

14 Handoff Checklist

When every box is ticked, the hardware side is done

Figure 13. End-to-end integration workflow



If any check fails: go to Troubleshooting (Ch. 13), fix, and repeat that stage.

A demonstration-mode number is never a blood-pressure measurement.

Eight stages from a bare board to a dataset the software team can use.

Checklist

☐ `rakshpulse doctor` → `all required checks passed`	Chapter 7
☐ Wiring documented: board model, breakout model, the pins and supply used	Chapter 5, lab notebook
☐ Sketch uploaded; Serial Monitor shows the `# rakshpulse ... fs_hz=...` banner and `index,red,ir` lines; monitor closed	Chapter 6, steps 9–10
☐ `rakshpulse check-serial --port <PORT> --fs <fs_hz>` → `PASS` (0 malformed lines, measured rate ≈ fs, no constant channel)	Chapter 9
☐ Lost-sample detection verified (deliberate `delay()` produces `missing_samples` > 0, then removed)	Chapter 9
☐ 5-minute bench recording with `record`; `inspect --validate` → all PASS; `missing_samples` < 0.1 %	Chapter 10, 12
☐ Removing the finger/ear mid-recording produces `missing` windows in `derived/windows.csv`	Chapter 12
☐ On-body recording at the target site: ≥ 80 % of resting windows `usable` (`inspect --quality`)	Chapter 12
☐ ≥ 3 cuff readings entered with `reference`; `analyze` reports them matched to windows	Chapter 11, 12
☐ `analyze` completes and writes `derived/predictions_<model>.csv` (expect `mode=demonstration`)	Chapter 12
☐ Zipped dataset folder sent privately to the software team	Below

Sharing the dataset

- Zip the **whole dataset folder** (for example `data/rakshpulse_hw`), not single files: the checksums in `dataset_manifest.csv` must travel with the data.
- Share it **privately** (an access-controlled drive or direct transfer). **Never commit participant recordings to the public repository**, and never post them in public channels.
- Subject IDs must be pseudonyms (`P001`). Keep the list linking pseudonyms to people separately and securely, and never send it with the data.
- Include a short note: board and breakout models, sketch settings, placement and mounting, cuff model, and anything unusual in any session.

POWERSHELL · zip on Windows

```
PS> Compress-Archive -Path data/rakshpulse_hw -DestinationPath rakshpulse_hw.zip
```

BASH · zip on Linux

```
$ zip -r rakshpulse_hw.zip data/rakshpulse_hw
```

✓ **EXPECTED RESULT: handoff complete**

When the checklist passes, the remaining work belongs to the software/ML side: more participants, repeat-day sessions with cuff readings, and training and evaluating a MAX30102-specific model.

15 Limitations and Next Steps

What is known, what is not, and what comes next

Limitations

Area	Limitation
Medical status	RakshPulse is a research prototype, not a medical device. No output is a clinically validated measurement.
Firmware	The reference sketch has not been compiled or run on hardware by the software team. Its FIFO-overflow handling follows the datasheet but is unverified on a board.
Hardware specifics	Pin numbers, supply voltage, pull-ups and LED current are board- and breakout-specific and are not specified by this manual.
Serial software	Tested with simulated sensors, replayed recordings and simulated (loop-back) serial ports, never with a physical MAX30102.
Model	No model is validated or experimental for MAX30102. The shipped model is demonstration-only on MAX30102 and was barely better than (SBP) or worse than (DBP) predicting the average on its own earlobe dataset.
Uncertainty	The shipped model's uncertainty interval is valid only for its earlobe training data; it is reported as unavailable for MAX30102.
Cross-day transfer	Project research found that the PPG contribution beyond a cuff reading did not transfer across days. Repeat-day data is needed to study this on MAX30102.
Protocol	This manual does not define a clinical measurement protocol; see `docs/DATA_COLLECTION_PROTOCOL.md` and your study's approved protocol.
Wireless	Only wired USB serial is supported in this release.

Next steps

#	Step	Owner	Done when
1	Compile and flash the firmware; pass `check-serial`	Hardware team	PASS on the target board
2	Verify lost-sample detection and tune `LED_POWER` at the target site	Hardware team	Gaps reported; ≥ 80 % usable at rest
3	Record sessions with ≥ 3 cuff readings each, several participants, at least two days each	Hardware team	Validated dataset shared privately
4	Build the feature table and train with cross-day validation (`features` → `train --split cross_day`)	Software team	Model compared with the baseline on unseen people and days
5	If it beats the baseline: ship it as `experimental` for MAX30102	Software team	Contract updated; outputs labelled `experimental`

#	Step	Owner	Done when
6	Independent evaluation against a recognised protocol before any `validated` claim	Future work	Not planned in this release

Where to find more

Document	Contents
`docs/HARDWARE_HANDOFF.md`	The short version of this manual, kept with the code
`hardware/firmware/rakshpulse_max30102/`	The reference sketch, with its configuration comments
`hardware/examples/`	Example serial streams (valid and malformed) and a simulated example dataset
`docs/DATA_COLLECTION_PROTOCOL.md`	Dataset format and recording protocol
`docs/USER_GUIDE.md`	Every command, including the ones the software team uses
`docs/RESEARCH_FINDINGS.md`	The research results behind the model status in Chapter 3

★ **IMPORTANT: the one-sentence summary**

Build it, prove the stream with `check-serial`, record carefully with cuff readings, validate, and share the dataset privately. Until a MAX30102 model is trained and evaluated, every BP number is a demonstration.